ChefMate — Beginner PRD and Step-by-Step Guide

Product Requirements Document (PRD)

ChefMate — Beginner Python Chatbot Project

Project level: BTech second-year beginner project
Application type: Python command-line chatbot
Development approach: Build independently using Python, a custom conversation dataset, a self-trained intent-classification model, local recipe data, and Python recipe-matching logic. No external generative-AI API, pretrained conversational chatbot, or chatbot API will be used.
Recommended first version: Python program with a JSON recipe file
Optional later version: SQLite database after the basic program works

---

1. Project Overview

ChefMate is a Python command-line chatbot that talks with the user and recommends recipes according to the user's answers. It asks conversational questions about available ingredients, meal type, cuisine, maximum cooking time, difficulty, and dietary preference. The program compares the collected information with recipes stored in a local file and displays the best matches.

In this project, the machine-learning component will classify the user's intention from a custom dataset created by the student. The recipe selection component will use Python rules, keyword extraction, filters, and scoring. This keeps the project self-developed and makes the model's behavior explainable.

Problem statement

People often have ingredients at home but do not know what they can prepare. Searching manually through many recipes takes time. ChefMate solves this problem through a simple conversation: it asks questions, collects the user's preferences, and provides suitable recipe suggestions from a stored recipe collection.

Main objective

To create a Python command-line chatbot that uses a self-trained intent-classification model to understand user messages, collects recipe preferences through conversation, matches them with stored recipes, ranks the results, and displays cooking instructions.

Scope decision

The first version should be small and reliable. It should train a lightweight intent-classification model on a custom dataset and recommend recipes from a prepared local dataset rather than inventing completely new recipes. A large language model and external AI API are outside the project scope.

---

2. Example User Experience

```
Welcome to ChefMate

Enter ingredients separated by commas: rice, tomato, onion
Choose meal type or press Enter for any: dinner
Maximum cooking time in minutes or press Enter for any: 30

Matching recipes:
1. Tomato Rice - 80% match - 25 minutes
```

```
2. Vegetable Rice - 60% match - 30 minutes

Enter a recipe number to view details, or 0 to start a new search: 1

Tomato Rice
Ingredients:
- Rice: 1 cup
- Tomato: 2
- Onion: 1

Steps:
1. Wash the rice.
2. Cook the onion and tomato.
3. Add rice and water.
4. Cook until ready.
```

---

3. Goals and Non-Goals

Goals

- Accept ingredients from the keyboard.
- Clean and normalize user input.
- Store recipes in a local JSON file.
- Match ingredients using understandable rules.
- Rank recipes according to a matching score.
- Support optional filters.
- Display recipe instructions.
- Handle invalid input without crashing.
- Keep the code simple enough for a beginner to explain.

Non-goals for the first version

- No external generative-AI API.
- No pretrained conversational chatbot.
- No large language model or text-generation model.
- No graphical interface.
- No web server.
- No login or user accounts.
- No online grocery ordering.
- No voice input.
- No medical or diet-treatment advice.
- No claim that allergy filtering is medically reliable unless the recipe data is verified carefully.

---

4. Minimum Viable Product

The first working version is complete when a user can:

1. Start the Python program from a terminal.
2. Load the trained intent-classification model.
3. Enter a natural-language message or answer a chatbot question.
4. Provide available ingredients.
5. Enter optional preferences or answer the model's intent questions.
6. See matching recipes ranked by score.

7. Select a recipe by number.
8. Read its ingredients and cooking steps.
9. Continue the conversation or request another recipe.
10. Receive a useful message when no recipe matches.
11. Exit the program safely.

Start with 10–20 clean recipes in `recipes.json`. Add more recipes only after the program works correctly.

---

5. Step-by-Step Development Plan

Each step begins with **What you should know first**. Complete the checkpoint before moving ahead.

Step 1: Understand the command line

What you should know first

Learn what a terminal or command prompt is, how to move between folders, and how to run a Python file. Learn the difference between a source-code file and the output printed by a program.

Work to complete

Practice these commands according to your operating system:

```
python --version
cd project-folder
python main.py
```

Create a file called `practice.py` that prints a welcome message.

Checkpoint

You should be able to open a terminal, move to your project folder, run a Python file, and understand the output.

Step 2: Learn the Python basics needed for this project

What you should know first

Learn variables, strings, integers, lists, dictionaries, conditions, loops, functions, and basic error handling.

Work to complete

Write small practice programs that:

- Store a user's name.
- Store a list of ingredients.
- Print each ingredient using a loop.
- Use a dictionary to store one recipe.
- Write a function that adds two numbers.
- Use an `if` statement to compare cooking times.

Checkpoint

You should be able to write a function that receives two ingredient lists and returns the number of common ingredients.

Step 3: Create the project folder

What you should know first

Understand why related files should be kept inside one project folder and why meaningful file names make a project easier to maintain.

Work to complete

Create this structure:

```
recipe_generator/
├── main.py
├── recipes.json
├── training_data.json
├── train_model.py
├── chatbot.py
├── recipe_logic.py
├── README.md
└── tests/
    └── test_recipe_logic.py
```

At the beginning, you may keep everything in `main.py` to understand the flow. Move matching functions to `recipe_logic.py` after the first version works.

Checkpoint

Run `main.py` from inside the project folder and print the project name successfully.

Step 4: Design the recipe data

What you should know first

Learn the difference between data and program logic. Recipe information belongs in a data file; the matching rules belong in Python code.

Work to complete

Create `recipes.json` with recipe records containing:

- `id`
- `name`
- `description`
- `meal_type`
- `cuisine`
- `difficulty`
- `prep_time`
- `cook_time`
- `diet_type`
- `ingredients`
- `steps`

Example:

```
[
  {
    "id": 1,
```

```
    "name": "Tomato Rice",
    "description": "A simple rice dish with tomato and onion.",
    "meal_type": "dinner",
    "cuisine": "indian",
    "difficulty": "easy",
    "prep_time": 10,
    "cook_time": 25,
    "diet_type": "vegetarian",
    "ingredients": [
      {"name": "rice", "quantity": "1 cup"},
      {"name": "tomato", "quantity": "2"},
      {"name": "onion", "quantity": "1"}
    ],
    "steps": [
      "Wash the rice.",
      "Cook the onion and tomato.",
      "Add rice and water.",
      "Cook until ready."
    ]
  }
]
```

Use consistent ingredient names. Choose either `tomato` or `tomatoes` and use the same form throughout the file.

Checkpoint

Create at least five recipes and confirm that the JSON file has valid syntax.

Step 5: Create the chatbot training dataset

What you should know first

Learn what supervised machine learning means: a model learns a relationship between example text and a label. In ChefMate, the labels are user intents such as `greeting`, `provide_ingredients`, `ask_recipe`, `choose_diet`, and `exit`. Also learn why training examples should use different wording for the same intent.

Work to complete

Create `training_data.json` with at least 10 intents and 15–25 example messages for each intent. Include different ways a user might express the same request. Keep a balanced number of examples for each intent.

Example:

```
[
  {"text": "Hello ChefMate", "intent": "greeting"},
  {"text": "What can I cook tonight?", "intent": "ask_recipe"},
  {"text": "I have rice and vegetables", "intent": "provide_ingredients"},
  {"text": "I want a vegetarian meal", "intent": "choose_diet"},
  {"text": "Goodbye", "intent": "exit"}
]
```

Checkpoint

Verify that every training message has both a `text` field and a correct `intent` label, and that no intent has far fewer examples than the others.

Step 6: Read the JSON files using Python

What you should know first

Learn how files are opened and how JSON is converted into Python lists and dictionaries. Learn why a file should be closed properly; using `with open(...)` handles this safely.

Work to complete

Write a function called `load_recipes()` that:

1. Opens `recipes.json`.
2. Reads the JSON data.
3. Returns the recipe list.
4. Displays a helpful message if the file is missing or invalid.

Print the recipe names to confirm that the data was loaded.

Checkpoint

Run the program and see all recipe names printed from the JSON file.

Step 7: Train the intent-classification model

What you should know first

Learn the purpose of training data, testing data, text vectorization, classification, accuracy, and a confusion matrix. Understand that the model predicts an intent; it does not automatically know cooking facts or create recipes.

Work to complete

Create `train_model.py`. Use a text vectorizer such as `TfidfVectorizer` and a beginner-friendly classifier such as `LogisticRegression` or `MultinomialNB`. Split the custom dataset into training and testing portions, train the model, print an evaluation report, and save the trained model and vectorizer locally.

Checkpoint

Test the model with messages that were not copied exactly from the training data. Record the accuracy and inspect incorrect predictions. Improve the dataset before changing the algorithm.

Step 8: Clean the user's ingredient input

What you should know first

Learn string methods such as `lower()`, `strip()`, and `split()`. Understand that `Rice`, `rice`, and `rice` should be treated as the same ingredient after normalization.

Work to complete

Create a function called `clean_ingredients()` that:

- Converts the input to lowercase.
- Splits the text at commas.
- Removes extra spaces.

- Removes empty entries.
- Returns a list of clean ingredient names.

Example:

```
Input:  Rice, tomato,  onion,
Output: ["rice", "tomato", "onion"]
```

Checkpoint

Test uppercase letters, extra spaces, repeated commas, and an empty input.

Step 9: Build the basic matching function

What you should know first

Understand lists, sets, common elements, and the difference between an ingredient the user has and an ingredient the recipe requires.

Work to complete

Create a function that receives a user ingredient list and one recipe. It should:

1. Extract the recipe ingredient names.
2. Find common ingredients.
3. Count the matches.
4. Calculate the recipe match percentage.
5. Return the result in a dictionary.

Use this formula:

```
match percentage = matching recipe ingredients / total recipe ingredients × 100
```

For example, if the user has four of a recipe's five ingredients, the match percentage is 80 percent.

Checkpoint

Test the function manually with exact matches, partial matches, and no matches.

Step 10: Add filters

What you should know first

Learn how multiple conditions work. Understand that an optional filter should be ignored when the user presses Enter without entering a value.

Work to complete

Add optional filters for:

- Meal type.
- Cuisine.
- Maximum total time.
- Difficulty.
- Dietary type.

Calculate total time as preparation time plus cooking time. Validate that a time value is a positive integer.

Checkpoint

Test each filter separately and then test combinations such as vegetarian recipes under 30 minutes.

Step 11: Rank the recipes

What you should know first

Learn sorting and understand that a score is only useful when its formula is documented.

Work to complete

Create a `find_matches()` function that:

1. Loads all recipes.
2. Applies selected filters.
3. Calculates a score for each remaining recipe.
4. Removes recipes with zero matching ingredients.
5. Sorts the recipes from highest score to lowest.
6. Returns the ranked results.

For the MVP, use the match percentage as the main score. Later, you may add a small bonus for an exact cuisine or meal-type match.

Checkpoint

Enter several ingredients and confirm that the recipe with the highest score appears first.

Step 12: Build the chatbot conversation flow

What you should know first

Learn how `input()` works and how a `while` loop can keep a program running until the user chooses Exit. Learn why each menu action should be handled by a separate function when possible.

Work to complete

Create a conversation flow such as:

```
ChefMate: Hello! What would you like to cook today?
User: I want a quick vegetarian dinner.
ChefMate: What ingredients do you have?
User: Rice, tomato, and onion.
ChefMate: I found these matching recipes...
```

The program should repeatedly show the menu, read the user's choice, and call the correct function. Reject choices such as letters or numbers outside the available options.

Checkpoint

You should be able to start the program, use every menu option, and exit without restarting the terminal.

Step 11: Display matching results

What you should know first

Learn how to loop over a list of dictionaries and format readable terminal output. Avoid printing one large block of confusing text.

Work to complete

Display each result with:

- Result number.
- Recipe name.
- Match percentage.
- Total time.
- Difficulty.
- Cuisine.
- Dietary type.

Example:

```
1. Tomato Rice
   Match: 80%
   Time: 35 minutes
   Difficulty: easy
```

If there are no results, explain that the user can add more ingredients or remove a filter.

Checkpoint

A user should understand why each recipe appeared without reading the source code.

Step 12: Add recipe detail selection

What you should know first

Learn how to validate a number selected by the user and how to retrieve the corresponding item from a list safely.

Work to complete

After displaying results, ask:

```
Enter a recipe number to view details, or 0 to return:
```

Show:

- Recipe name.
- Description.
- Ingredients and quantities.
- Preparation time.
- Cooking time.
- Difficulty.
- Ordered steps.

Reject invalid numbers and keep the user in the menu instead of crashing.

Checkpoint

Open at least three recipe details and verify that each recipe shows its own correct information.

Step 13: Add validation and error handling

What you should know first

Learn that users may enter empty, unexpected, or invalid values. Learn the difference between handling an invalid input and hiding a programming error.

Work to complete

Handle these cases:

- Empty ingredient input.
- Input containing only commas or spaces.
- Invalid time such as `abc`.
- Negative time.
- Invalid menu choice.
- Invalid result number.
- Missing `recipes.json`.
- Incorrect JSON syntax.

Use short, friendly messages. During development, inspect technical errors; during normal use, do not show confusing error traces.

Checkpoint

Deliberately enter incorrect values and confirm that the program continues running.

Step 14: Refactor the code

What you should know first

Understand that refactoring means improving the structure without changing the program's intended behavior. Learn why repeated code should be placed in functions.

Work to complete

Separate the program into functions such as:

```
load_recipes()
clean_ingredients()
get_user_preferences()
calculate_match()
find_matches()
display_results()
display_recipe_details()
show_main_menu()
```

Use clear variable names and short comments for logic that is not obvious. Keep the main program flow easy to read.

Checkpoint

You should be able to explain the purpose of every function in one sentence.

Step 15: Write tests for the matching logic

What you should know first

Learn what a test case, expected result, and assertion are. Test the small matching functions separately from the interactive menu.

Work to complete

Test:

- Exact ingredient matching.
- Partial ingredient matching.
- Case differences.
- Extra spaces.
- Empty input.
- No matching ingredients.
- Filter matching.
- Match-score calculation.
- Sorting by score.
- Invalid time input.

You may begin with simple manual test cases and then use Python's `unittest` module.

Checkpoint

All core matching tests should pass after you test both normal and incorrect inputs.

Step 16: Improve the command-line experience

What you should know first

Learn that a CLI can still be user-friendly. Clear prompts, spacing, consistent wording, and predictable navigation are more important than decoration.

Work to complete

Add:

- A clear welcome message.
- Numbered menus.
- Consistent headings.
- Blank lines between sections.
- A return-to-menu option.
- A reset-search option.
- A final exit message.
- Optional terminal colors only after the plain version works.

Do not add colors or external libraries until the basic output is correct.

Checkpoint

Ask another person to use the program without instructions. Record where they become confused and improve those prompts.

Step 17: Document the project

What you should know first

Understand that a college project is evaluated by its explanation as well as its code. Your documentation should show what you built, how it works, how you tested it, and what it cannot do.

Work to complete

Prepare:

- Abstract.
- Problem statement.
- Objectives.
- Scope.
- Feasibility study.
- Functional requirements.
- Non-functional requirements.
- Program flowchart.
- Data-file structure.
- Algorithm explanation.
- Screenshots of terminal output.
- Test cases.
- Limitations.
- Future enhancements.
- Conclusion.
- README with installation and usage instructions.

Keep a development diary containing dates, concepts learned, errors encountered, and solutions.

Checkpoint

Another student should be able to install and run the project using only your README.

Step 18: Prepare the demonstration

What you should know first

A good demonstration should show the problem, input, decision process, and output. Do not only show the program's welcome screen.

Work to complete

Demonstrate this sequence:

1. Start the program.
2. Explain the problem in one minute.
3. Enter available ingredients.
4. Apply one filter.
5. Show the ranked results.

6. Open a recipe's details.
7. Demonstrate a no-result case.
8. Explain the matching percentage.
9. Explain limitations and future improvements.

Keep screenshots or a short backup recording in case the program does not run during the presentation.

Checkpoint

Practice until you can explain the program and its algorithm without reading every line of code.

---

6. Functional Requirements

- **FR-01**: The program shall start from a terminal using Python.
- **FR-02**: The program shall display a main menu.
- **FR-03**: The program shall accept comma-separated ingredients.
- **FR-04**: The program shall normalize ingredient input.
- **FR-05**: The program shall allow optional filters.
- **FR-06**: The program shall read recipes from a local data file.
- **FR-07**: The program shall calculate a recipe matching score.
- **FR-08**: The program shall rank matching recipes.
- **FR-09**: The program shall display recipe summaries.
- **FR-10**: The program shall display complete recipe details.
- **FR-11**: The program shall handle invalid input.
- **FR-12**: The program shall provide a no-results message.
- **FR-13**: The program shall allow the user to start another search.
- **FR-14**: The program shall exit safely.

7. Non-Functional Requirements

- The program should be understandable to a beginner developer.
- The program should run on a normal computer with Python installed.
- The program should not require an internet connection after setup.
- The program should not store passwords or sensitive personal data.
- The program should use clear prompts and readable output.
- The program should use functions instead of one very long block of code.
- The program should not crash because of ordinary invalid input.
- The recommendation score should be explainable.

8. Program Flow

```
Start
  ↓
Load recipes from recipes.json
  ↓
Display main menu
  ↓
Read user choice
  ↓
If Find Recipe:
    Read ingredients and filters
    Clean input
```

```
    Match recipes
    Calculate scores
    Sort results
    Display results
    Show selected recipe details
  ↓
If View All Recipes:
    Display recipe names
  ↓
If Exit:
    Close program
  ↓
Return to menu
```

9. Basic Matching Algorithm

```
1. Read the user's ingredient text.
2. Convert it to lowercase.
3. Split it at commas.
4. Remove spaces and empty entries.
5. Read all recipes from the JSON file.
6. Apply selected filters.
7. Compare the user's ingredients with each recipe's ingredients.
8. Count matching ingredients.
9. Calculate the match percentage.
10. Remove recipes with zero matches.
11. Sort recipes from highest score to lowest.
12. Display the results.
```

The scoring rule should remain visible in your documentation. If you later add bonuses for cuisine or meal type, explain how those bonuses affect the final score.

10. Common Beginner Mistakes to Avoid

- Starting with machine learning before building the rule-based version.
- Building a graphical interface when the project requirement is command line.
- Adding too many features before testing the basic search.
- Keeping all logic in one enormous function.
- Using inconsistent ingredient names.
- Forgetting to validate `input()` values.
- Assuming a JSON file is always valid.
- Copying code without understanding it.
- Treating the recommendation as medical or allergy advice.
- Waiting until the final day to test the program.

11. Recommended Learning Order

1. Terminal commands.
2. Python syntax.
3. Lists and dictionaries.
4. Conditions and loops.
5. Functions.
6. Reading and writing files.

7. JSON.
8. Error handling.
9. Sorting and scoring.
10. Testing with `unittest`.
11. Git and README writing.
12. SQLite as an optional improvement.

Use classroom notes, official Python documentation, beginner books, and small practice exercises. When studying an example, close it and rewrite the idea in your own words and structure.

12. Future Enhancements

After the command-line MVP is stable, possible additions include SQLite storage, saved favorite recipes, user profiles, nutrition information, ingredient substitutions, multilingual output, a graphical interface, a web version, and a machine-learning recommendation model. These are future enhancements and are not required for the first submission.

13. Definition of Done

The project is ready for submission when:

- It starts from the terminal without a critical error.
- The user can search using ingredients.
- Optional filters work correctly.
- Results are ranked using documented rules.
- Recipe details are displayed correctly.
- Invalid input is handled without crashing.
- No-result behavior is handled.
- The JSON file contains clean sample recipes.
- Core matching functions have tests.
- The README explains installation and usage.
- Terminal screenshots and test evidence are included in the report.
- The developer can explain every major part of the code.

Final advice

Build the smallest working version first: one Python file, five recipes, one matching function, and one menu. Then add filters, more recipes, refactoring, testing, and documentation one step at a time. Do not add another feature until you understand the current one. The goal is not only to submit a command-line application; it is to understand how every part of the application works.